Chapter 26: Maximum Flow

1) Flow Networks: A flow network $G = (V, E)$ is a directed graph in which each edge $(u, v) \in E$ has a non-negative capacity $c(u, v) \geq 0$. If $(u, v) \notin E$ we assume $c(u, v) = 0$. Source (s) is a vertex that produces some material at a constant rate. Sink (t) is a vertex that consumes the material at the same rate.

For convenience we assume that every vertex lies on some path from the source to the sink. That is, for every vertex $v \in V$, there is a path $s \rightsquigarrow v \rightsquigarrow t$. The graph is therefore connected, and $|E| \geq |V| - 1$.

→ **Flow:** Let $G = (V, E)$ be a flow network with a capacity function c. Let s be the source of the network, and let t be the sink. A "flow" in G is a real-valued function $f : V \times V \to R$ that satisfies the following three properties:—

i) **Capacity constraint:** For all $(u, v) \in V$, we require $f(u, v) \leq c(u, v)$.

ii) **Skew symmetry:** For all $(u, v) \in V$, we require $f(u, v) = -f(v, u)$.

iii) **Flow conservation:** For all $u \in V - \{s, t\}$, we require $\sum_{v \in V} f(u, v) = 0$.

→ The quantity $f(u, v)$ is called the flow from vertex u to vertex v, and can be positive, zero or negative.

The value of a flow f is defined as

$|f| = \sum_{v \in V} f(s,v)$ i.e. the total flow out of the source

→ In the "maximum-flow problem", we are given a flow network G with source s and sink t, and we wish to find a flow of maximum value.

→ The capacity constraint simply says that the flow from one vertex to another must not exceed the given capacity. Skew symmetry is a notational convenience which says that the flow from a vertex u to a vertex v is the negative of the flow in the reverse direction. The flow-conservation property states that the total flow out of a vertex other than source or sink is 0.

→ By skew-symmetry we can rewrite the flow-conservation property as:

$\sum_{u \in V} f(u,v) = 0$ for all $v \in V - \{s,t\}$. That is, the total flow into a vertex is 0.

When neither (u,v) nor (v,u) is in E, there can be no flow between u and v and $f(u,v) = f(v,u) = 0$.

→ The "total positive flow entering a vertex" v is defined by $\sum_{\substack{u \in V \\ f(u,v) > 0}} f(u,v)$.

The total positive flow leaving a vertex u is defined by $\sum_{\substack{v \in V \\ f(u,v) > 0}} f(u,v)$.

The "total net flow" at a vertex is defined to be the total positive flow leaving a vertex minus the total positive flow entering a vertex.

→ Flow networks can be used to model liquids flowing through pipes, parts through assembly lines, current through electrical networks, information through communication networks etc.

→ Suppose a crates shipment problem. Suppose 2 vertices Edmonton and Calgary. A company may decide to ship 8 crates per day from Edmonton (v_1) to Calgary (v_2) and 3 crates per day from Calgary (v_2) to Edmonton (v_1). You cannot represent these shipments directly by flows i.e. you cannot allow $f(v_1, v_2) = 8$ and $f(v_2, v_1) = 3$ since skew symmetry requires $f(v_1, v_2) = -f(v_2, v_1)$.

You need to make use of cancellation and let $f(v_1, v_2) = 5$ and $f(v_2, v_1) = -5$ i.e. 5 crates from Edmonton to Calgary and "0" crates from Calgary to Edmonton.

Typically, we shall not care about how the actual shipments are set up; for any pair of vertices, we only care about the net amount that travels between them. If we do care about the underlying shipments, then we should be using a different model, one that retains information about shipments in both directions.

2) Networks with multiple sources and sinks:

We can reduce the problem of determining a maximum flow in a network with multiple sources and multiple sinks to an ordinary maximum-flow problem. We add a supersource s and add a directed edge (s, s_i) with capacity $c(s, s_i) = \infty$ for each $i = 1, 2, \ldots, m$. We also create a new supersink t and add a directed edge (t_i, t) with capacity $c(t_i, t) = \infty$ for each $i = 1, 2, \ldots, n$ [n sources m sinks, initially].

3) Usually we deal with several functions (like f) that take as argument 2 vertices in the flow network. In this chapter, we shall use an "implicit summation notation" in which either argument, or both may be a set of vertices. For example:-

$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y) .$$

We may also omit set braces. For example $f(s, V - s)$ implies $f(s, V - \{s\})$.

The flow-conservation constraint can be expressed as:- $f(u, V) = 0$ for all $u \in V - \{s, t\}$.

Lemma

Let $G = (V, E)$ be a flow network, and let f be a flow in G. Then the following equalities hold:-

1) For all $X \subseteq V$, we have $f(X, X) = 0$.

2) For all $X, Y \subseteq V$, we have $f(X,Y) = -f(Y,X)$.

3) For all $X, Y, Z \subseteq V$ with $X \cap Y = \phi$, we have the sums $f(X \cup Y, Z) = f(X,Z) + f(Y,Z)$ and $f(Z, X \cup Y)$ $= f(Z,X) + f(Z,Y)$.

We have :-

$|f| = f(s,V)$ (by definition)

$= f(V,V) - f(V-s,V)$ (by Lemma, part 3)

$= -f(V-s,V)$ (by Lemma, part 1)

$= f(V, V-s)$ (" " , part 2)

$= f(V,t) + f(V, V-s-t)$ (" " , part 3)

$= f(V,t)$ (by flow-conservation)

4) The Ford-Fulkerson method :

The basic Ford-Fulkerson method is as follows:

FORD-FULKERSON-METHOD(G,s,t)

1 initialize flow f to 0 i.e. for every edge $(u,v) \in E$ let $f(u,v) = 0$

2 while there exists an augmenting path p

3 do augment flow f along p

4 return f

An augmenting path is simply a path from the source s to the sink t along which we can

send more flow.

Residual networks

The residual capacity of an edge (u,v) is given by

$$c_f(u,v) = c(u,v) - f(u,v)$$

and is the maximum amount of additional flow we can push from u to v without exceeding the capacity.

Given a flow network $G = (V,E)$ and a flow f, the residual network of G induced by f is $G_f = (V, E_f)$, where

$$E_f = \{(u,v) \in V \times V : c_f(u,v) > 0\}.$$

Each edge of the residual network is called residual edge and can admit a positive flow.

The edges in E_f are either edges in E or their reversals. Corresponding to any edge (u,v) in E there can be at max two edges in E_f:

(i) If $f(u,v) < c(u,v)$ then $c_f(u,v) = c(u,v) - f(u,v) > 0$ and $(u,v) \in E_f$. If $f(u,v)$ equals $c(u,v)$ the residue is 0 and $(u,v) \notin E_f$.

(ii) If $f(u,v) > 0$ then skew-symmetry implies that $f(v,u) < 0$. $c_f(v,u) = c(v,u) - f(v,u) = 0 - f(v,u) > 0$. Hence, $(v,u) \in E_f$.

If neither (u,v) nor (v,u) appears in the original network, then they can also not appear in the residual network. Thus,

$$|E_f| \leq 2|E|$$

The residual network itself is a flow network with capacities given by c_f.

Lemma

Let $G = (V, E)$ be a flow network with source s and sink t, and let f be a flow in G. Let G_f be the residual network of G induced by f, and let f' be a flow in G_f. Then the "flow sum" $f + f'$ is a flow in G with value $|f + f'| = |f| + |f'|$

Proof: We must verify that skew symmetry, capacity constraint and flow-conservation are obeyed.

(i) For skew-symmetry, note that for all $u, v \in V$, we have:

$$\begin{aligned}(f+f')(u,v) &= f(u,v) + f'(u,v) \\ &= -f(v,u) - f'(v,u) = -[f(v,u) + f'(v,u)] \\ &= -(f+f')(v,u)\end{aligned}$$

(ii) For the capacity constraints, for all $u, v \in V$ we have:-

$$\begin{aligned}(f+f')(u,v) &= f(u,v) + f'(u,v) \\ &\leq f(u,v) + c_f(u,v) \\ &= f(u,v) + c(u,v) - f(u,v) \\ &= c(u,v)\end{aligned}$$

(iii) For flow conservation, note that for all $u \in V - \{s,t\}$, we have:-

$$\sum_{v \in V} (f+f')(u,v) = \sum_{v \in V} (f(u,v) + f'(u,v))$$

$$= \sum_{v \in V} f(u,v) + \sum_{v \in V} f'(u,v) \quad = 0 + 0 = 0$$

Finally, we have:-

$$|f + f'| = \sum_{v \in V} (f + f')(s,v) \quad = \sum_{v \in V} (f(s,v) + f'(s,v))$$

$$= \sum_{v \in V} f(s,v) + \sum_{v \in V} f'(s,v) \quad = |f| + |f'|$$

→ Augmenting paths:

Given a flow network $G = (V, E)$ and a flow f, an augmenting path p is a simple path from s to t in the residual network G_f. Each edge (u,v) on an augmenting path can admit some additional positive flow from u to v without violating the capacity constraints on the edge.

The "residual capacity" of an augmenting path is the maximum amount by which we can increase the flow on each edge of the augmenting path, and is given by $c_f(p) = \min\{c_f(u,v) : (u,v) \text{ is on } p\}$.

Lemma

Let $G = (V, E)$ be a flow network, let f be a flow in G and let p be an augmenting path in G_f. Define a function $f_p : V \times V \to R$ by

$$f_p(u,v) = \begin{cases} c_f(p) & \text{if } (u,v) \text{ is on } p \\ -c_f(p) & \text{if } (v,u) \text{ is on } p \\ 0 & \text{otherwise} \end{cases}$$

Then, f_p is a flow in G_f with value $|f_p| = c_f(p) > 0$.

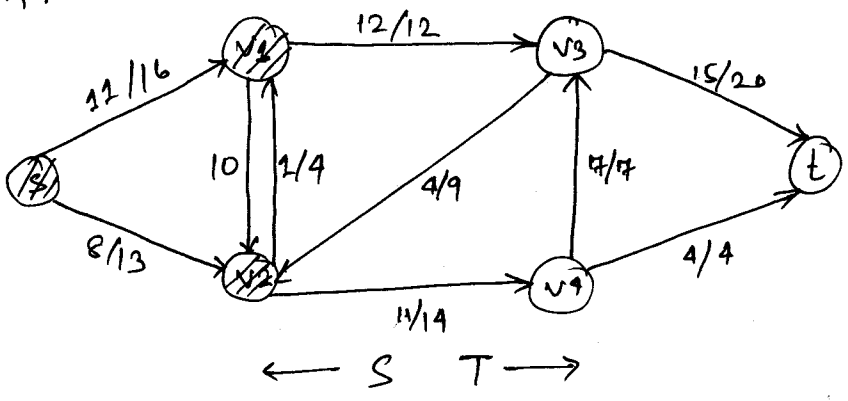
Corollary

Let $G = (V, E)$ be a flow network, let f be a flow in G, and let p be an augmenting path in G_f. Define a function $f' : V \times V \to R$ by $f' = f + f_p$. Then f' is a flow in G with value $|f'| = |f| + |f_p| > |f|$.

→ Value of $f_p + f$ is closer to the maximum flow than f.

→ Cuts of flow-networks :

A cut (S, T) of a flow-network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$. The "net flow" across a cut is defined to be $f(S, T)$. The capacity of the cut (S, T) is $c(S, T)$. A "minimum cut" of a network is a cut whose capacity is minimum over all cuts of the network.



a/b means flow/capacity. A single value means just capacity.

$$f(S, T) = f(V_1, V_3) + f(V_2, V_3) + f(V_2, V_4)$$

$$= 12 - 4 + 11 = 19$$

and its capacity is

$$c(S, T) = c(V_1, V_3) + c(V_2, V_4) = 12 + 14$$

$$= 26$$

Net flow across a cut (S,T) consists of positive flows in both directions: positive flow from S to T is added while positive flow from T to S is subtracted.
On the other hand, the capacity of a cut (S,T) is computed only from edges going from S to T.

→ The net flow across any cut is the same and it equals the value of the flow:-

Lemma

Let f be a flow in a flow network G with source s and sink t, and let (S,T) be a cut of G. Then the net flow across (S,T) is $f(S,T) = |f|$.

Proof:

$$\begin{aligned} f(S,T) &= f(S,V) - f(S,S) \\ &= f(S,V) \\ &= f(s,V) + f(S-s,V) \\ &= f(s,V) \\ &= |f| \end{aligned}$$

Corollary

The value of any flow f in a flow network G is bounded from above by the capacity of any cut of G.

Proof: Let (S,T) be any cut of G and let f be any flow.

$$\begin{aligned} |f| = f(S,T) &= \sum_{u \in S} \sum_{v \in T} f(u,v) \\ &\leq \sum_{u \in S} \sum_{v \in T} c(u,v) = c(S,T) \end{aligned}$$

→ Max-flow min-cut theorem states that the value of a maximum flow equals the capacity of a minimum cut.

Max-flow min-cut theorem

If f is a flow in a flow network $G = (V, E)$ with source s and sink t, then the following conditions are equivalent:

1) f is a maximum flow in G.
2) The residual network G_f contains no augmenting paths.
3) $|f| = c(S, T)$ for some cut (S, T) of G.

The basic Ford-Fulkerson algorithm

FORD_FULKERSON (G, s, t)

```
1  for each edge (u,v) ∈ E[G]
2      do f[u,v] ← 0
3         f[v,u] ← 0
4  while there exists a path p from s to t in the
   residual network G_f
5      do c_f(p) ← min { c_f(u,v) : (u,v) is in p }
6         for each edge (u,v) in p
7             do f[u,v] ← f[u,v] + c_f(p)
8                f[v,u] ← -f[u,v]
```

(We use square brackets when we treat an identifier -such as f- as a mutable field, and we use parentheses when we treat it as a function.)

→ Analysis of Ford-Fulkerson

The running time of FORD-FULKERSON depends on how the augmenting path is determined. If it is chosen poorly, the algorithm might not even terminate: the value of the flow will increase with successive augmentations, but it need not even converge to the maximum flow value. Ford-Fulkerson method might fail to terminate only if edge capacities are irrational numbers. However, in practice, irrational numbers cannot be stored on finite-precision computers.

Most often, the maximum-flow problem arises with integral capacities. If capacities are rational numbers, an appropriate scaling transformation can make them all integral. Under this assumption, a straightforward implementation of FORD_FULKERSON runs in time $O(E|f^*|)$, where f^* is the maximum flow found by the algorithm: Lines 1,2,3 take time $\Theta(E)$. The while loop is executed at most $|f^*|$ times, since the flow value increases by at least one unit in each iteration.

What is the running time of a single iteration of the while loop? Work done by the while loop can be made efficient if we efficiently manage the data structure used to implement the network

$G = (V, E)$. Let us assume that we keep a data structure corresponding to a directed graph $G' = (V, E')$ where $E' = \{(u,v) : (u,v) \in E \text{ or } (v,u) \in E\}$. Edges in G are also edges in G' and it is therefore a simple matter to maintain capacities and flows in this data structure. The edges in the residual network G_f consist of all edges (u,v) of G' such that $c[u,v] - f[u,v] \neq 0$. The time to find a path in a residual network is therefore $O(V + E') = O(E)$ if we use either breadth-first search or depth-first search. Each iteration of the while loop takes $O(E)$ time, making the total running time of FORD-FULKERSON $O(E|f^*|)$.

5) The Edmonds-Karp algorithm :

The bound on FORD-FULKERSON can be improved if we implement the computation of the augmenting path with a breadth-first search. The augmenting path then would be a shortest path from s to t in the residual network, where each edge has unit distance (weight). The ford-fulkerson method so implemented is called Edmonds-Karp algorithm and runs in time $O(VE^2)$.

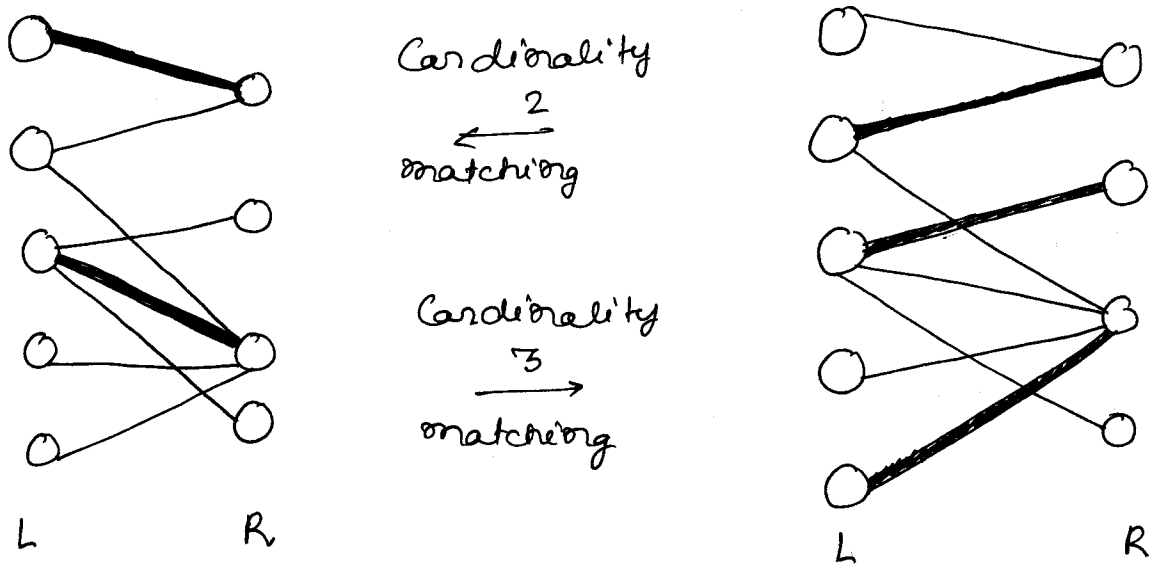
6) Maximum bipartite matching

→ Given an undirected graph $G = (V, E)$, a "matching" is a subset of edges $M \subseteq E$ such that for all vertices $v \in V$, at most one edge of M

is incident on v. We say that a vertex $v \in V$ is "matched" by matching M if some edge in the matching M is incident on v. A vertex on which no edge of a matching is incident is said to be "unmatched".

→ A "maximum matching" is a matching of maximum cardinality. There may be more than one maximum matching for an undirected graph.

→ A vertex set of a bipartite graph can be partitioned into 2 disjoint sets i.e. $V = L \cup R$ and $L \cap R = \phi$. Every edge in E goes between L and R. We assume that every vertex in V has at least one incident edge. Here we shall restrict attention to finding maximum matchings in bipartite graphs.



→ The problem of finding a maximum matching in a bipartite graph has many practical applications. The set L could be a set of machines and the set R could be a set of tasks to be performed simultaneously. An edge (u,v) in E implies that machine u can perform

task v. A maximum matching provides work for as many machines as possible.

→ Finding a maximum bipartite matching

We can use the Ford-Fulkerson method to find a maximum matching in an undirected bipartite graph $G = (V, E)$. First of all we construct a flow network corresponding to a bipartite graph, in which flows correspond to matchings.

For a bipartite graph G we define the corresponding flow network $G' = (V', E')$ where $V' = V \cup \{s, t\}$ i.e. we add 2 new vertices to the existing graph : a source s and a sink t.

$$E' = E \cup \{(s, u) : u \in L\} \cup \{(v, t) : v \in R\}.$$

Each edge in E' is assigned a unit capacity.
Since each vertex in V has at least one incident edge and because a single edge spans 2 vertices we have :-

$|E| \geq |V|/2 \Rightarrow |V| \leq 2|E|$

$|E'| = |E| + |V|$ [We add V new edges to E to produce E']

$\leq 3|E|$

also $|E| \leq |E'|$

Hence, $|E'| = \Theta(E)$.